

1. What are the key implementation/optimization techniques for OLTP and OLAP?

OLTP systems are built for speed. They rely on row-oriented storage, writing entire records in one shot so inserts and updates never lag. Row-level locking and concurrency control let hundreds of users operate at once without crashing into each other, keeping transactions safe and isolated. They lean on B-tree indexes to pull up exactly what's needed in an instant, and they have a write-optimized architecture, so they commit fast through lightweight logging that preserves durability with minimal overhead. Even the cost-based optimizer is tuned for short, predictable queries.

OLAP, on the other hand, doesn't chase quick transactions but digs deep into the past, pulling patterns from millions of rows. Here, column-oriented storage saves time by grabbing only the attributes needed instead of hauling entire rows. Bitmap indexes shrink massive datasets into something manageable, while star schemas with fact and dimension tables lay out the world in a way that makes sense for analysis. Materialized views stand like shortcuts, pre-computed answers waiting to be used again. OLAP is tuned for large sequential scans and heavy aggregation, not quick writes. It isn't about speed in the moment—it's about clarity, about reading the past to guide the future.

2. What are the challenges for stream processing? Why existing relational DBMS cannot handle it well?

Stream processing faces several challenges, one of which is that data must be handled in real-time, processed as it arrives rather than stored first, with latency measured in milliseconds. Unlike static tables, streams are unbounded, so systems need mechanisms such as sliding or tumbling windows to partition the flow into meaningful chunks. Another one of its challenges is

throughput. Financial trading or sensor monitoring can generate hundreds of thousands of events per second, requiring engines that scale accordingly. Streams also arrive with irregularities: events may be late, missing, or out of order, and the system must handle these cases without breaking correctness guarantees. Finally, stream systems must integrate application logic alongside queries—such as alert generation or real-time decision rules—something that traditional query engines were never designed to handle.

Existing relational DBMSs cannot handle these challenges well. Their foundation is the outbound model: data must be stored before it can be queried, which introduces unnecessary overhead and delays. SQL lacks the right basis and foundation. There is no native way to express windows or handle late or out-of-order data without creating complex ways to get around it. The DBMS architecture is also another one of the reasons relational DBMSs can't handle these challenges. The client-server separation applies process boundaries between the query engine and application code, leading to context-switching overhead. And the reliance on full ACID guarantees—heavyweight logging, strict locking, and recovery mechanisms—slows down workloads where lighter synchronization or approximate answers would be sufficient. In short, relational DBMSs are optimized for durability and consistency, rather than for continuous, high-throughput, low-latency flows, which is stream processing.

3. Why high availability requirement has conflicts with ACID transaction management?

High availability requirements often have conflicts with ACID transaction management because ACID durability relies on mechanisms like write-ahead logging and replay. After a crash, the database must scan logs and restore state precisely. This can take seconds or even minutes.

During this time, the system is unavailable. This process maximizes correctness but sacrifices uptime.

On the other hand, high availability focuses on continuous service. In real-time or streaming applications, downtime cannot exist because even short pauses can delay operations. High availability systems pull this off using process pairs, replication, or hot standbys. These processes allow instant failover. However, these methods often relax ACID guarantees which allow lost or duplicated messages for the sake of uninterrupted service. In conclusion, ACID applies strict recovery semantics, while high availability focuses on responsiveness.